Agentic Context Engineering (ACE)

A framework from Stanford, SambaNova Systems & UC Berkeley that enables coding agents to self-improve from real engineering experience, without retraining the model.

Presented by: Mubeen Bhatti

Problem Statement

Coding agents have no persistent memory of what worked, or what failed.

Modern AI agents are forced to rewrite their entire memory every time they learn something new. Like an employee who must rewrite their entire training manual just to add one new rule, they inevitably compress, summarise, and lose the critical nuance that makes them effective.

- Agents repeat the same mistakes across every run: no accumulated learning
- Prompt optimisers suffer from brevity bias, compressing away crucial domain-specific detail
- Monolithic context rewrites trigger context collapse, degrading accumulated knowledge
- Retraining the base model is expensive, slow, and does not resolve the operational knowledge gap

Value Proposition : ACE Framework

Generator

The junior developer. Reads tickets, inspects the repository, proposes changes, and executes tools, interacting with compilers, test runners, APIs, and sandboxes.

Reflector

The QA tester. Reviews execution logs after each task, analyses environment feedback and error codes, and extracts concrete, actionable insights.

Example: If an API returns paginated results of 100 rather than a full list, the Reflector identifies the pagination gap and flags it.

Curator

The lead architect. Takes the Reflector's insight and integrates it into the master playbook as a single, localised bullet point, without touching or rewriting the rest of the document.

This is how ACE prevents context collapse.

By dividing cognitive labour across three roles, ACE bypasses the bottleneck of overloading a single process with execution, critique, and synthesis, mirroring how high-performing engineering teams actually operate.

Generator vs. Reflector: Complementary Roles in ACE

In the ACE framework, the Generator and the Reflector serve different but complementary functions. The Generator is the "doer" that attempts tasks, while the Reflector is the "critic" or "diagnostician" that analyzes what went wrong and extracts lessons.

Generator — The Executor

Takes the user's query and the current context playbook, produces a reasoning trajectory, calls external tools, and generates an output. Leaves behind an execution trace that exposes its strategies, decisions, and any mistakes it made along the way.

> Like a student taking a math test — uses the study guide (playbook) to attempt a complex equation, writes down step-by-step work (the reasoning trace), and sometimes applies the wrong formula. The job is simply to execute the task and show the work.

Reflector — The Diagnostician

After the Generator produces its execution trace, the Reflector steps in to analyze it. It identifies exactly where and why the Generator went wrong — misapplied rules, flawed reasoning steps, or incorrect assumptions — and distills these into specific, actionable lessons. It does NOT fix the problem itself; it diagnoses and extracts the insight.

> Like a teacher grading the student's test — reviews the step-by-step work, circles the exact mistake (e.g., "You forgot to carry the 1 in step 3"), and writes a clear correction note. The teacher doesn't retake the test; they identify the lesson to be learned.

The Generator tries to accomplish the task and produces the raw experience. The Reflector reviews that experience, pinpoints what went wrong, and extracts the precise lesson — ready to be filed into memory.

ACE in Action: Ordering Food at a Restaurant

The three ACE roles are easy to understand through a simple, everyday scenario — going to a restaurant.

1

The Generator: Doing the Task

The Generator is like you ordering a meal. You pick a dish based on what you think you'll like — maybe you try something new, like a spicy pasta. This is your first attempt at solving the problem (hunger) using your current knowledge.

TRY A SOLUTION

2

The Reflector: Learning from What Happened

The Reflector is like reflecting after the meal: "This was too spicy." "The portion was small." "The service was slow." You identify what worked and what didn't — extracting specific, actionable lessons from the experience.

ANALYZE THE OUTCOME

3

The Curator: Saving What Matters

The Curator is like remembering those lessons next time you go out: avoid very spicy dishes, order larger portions, choose faster restaurants. Over time, your dining decisions get better and better.

STORE LESSONS FOR NEXT TIME

Instead of repeating the same mistakes every time you go out, ACE builds a growing "personal dining playbook" — each experience makes the next decision smarter.

ACE Playbook Advantage

Most AI memory systems fail because they treat memory as a single document to rewrite. ACE treats it as a structured, modular knowledge base: entries are appended, refined, or retired individually, never overwritten wholesale.

Modular by Design

Individual entries added or removed without touching the rest of the playbook.

Machine Unlearning in One Step

Delete a bullet point. The model instantly unlearns, with zero retraining required.

Incremental Delta Updates

New knowledge merged via lightweight logic: no full rewrites, no context collapse.

Grow and Refine

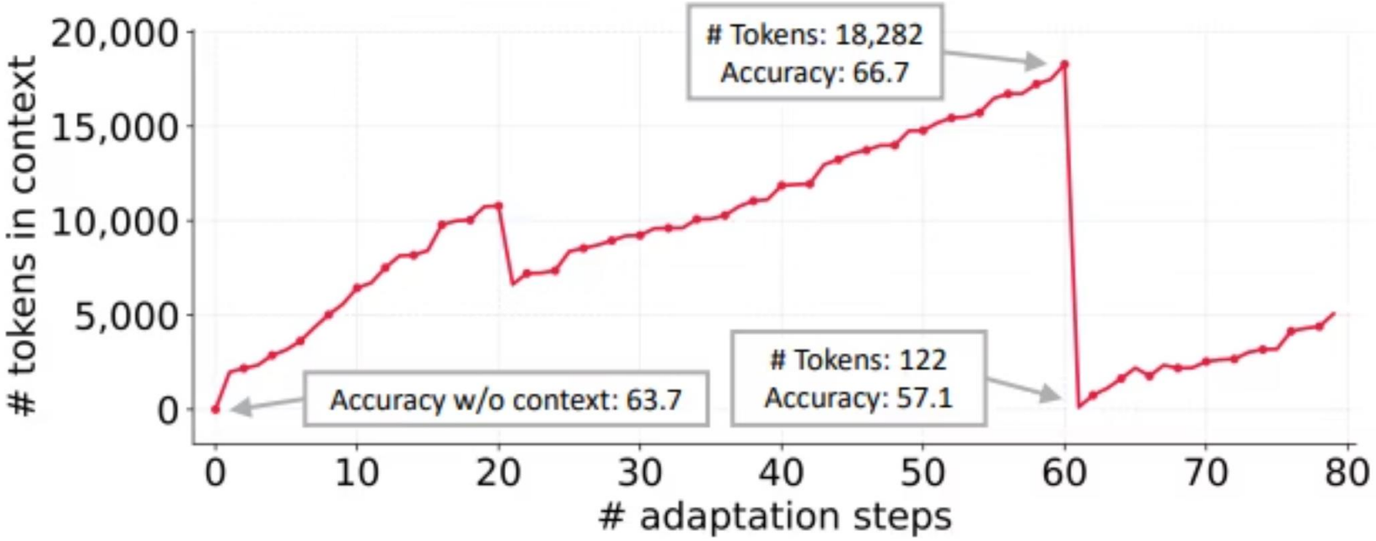
Semantic clustering merges similar rules: preserving nuance, not compressing it.

Human-Readable and Auditable

Plain structured text: any engineer or compliance officer can read, verify, or delete entries instantly.

Limitations of Existing Context Adaptation Methods

Context collapse is a fundamental risk of monolithic memory rewrites.



In an experiment, a single rewrite compressed 18,282 tokens to 122, dropping accuracy from 66.7% to 57.1% below the 63.7% baseline of an empty context. LLMs are trained to synthesis: when forced to rewrite memory, they discard the nuance that makes agents effective.

Where ACE Fits Relative to Other Frameworks

Reflexion

Improves agents through verbal feedback and episodic memory, but does not produce structured, curated enterprise playbooks.

MIPRO

A strong prompt optimiser for multi-stage LM programs, but suffers from brevity bias: it systematically compresses away domain-specific nuance.

Dynamic Cheatsheet

The direct conceptual predecessor to ACE. ACE addresses its core weakness, monolithic memory rewrites, through structured delta updates and grow-and-refine curation.

CodeAct

Demonstrates the value of execution-grounded agents. ACE complements it by providing the governed, evolving memory layer that CodeAct lacks.

The simplest positioning: CodeAct provides an action-capable agent. ACE gives that agent a governed, self-improving memory without touching the model weights.

ACE Relative Performance

Using a smaller open-source model (DeepSeek V3), ACE matched IBM's top-ranked production agent, powered by GPT-4.1, on the AppWorld* leaderboard, and outperformed it on the hardest test-challenge split. It achieved this at a fraction of the compute cost.

10.6%

Agent Benchmark Gains

Average improvement on the AppWorld benchmark vs. baseline methods

8.6%

Finance Task Gains

Accuracy boost on XBRL-structured financial filings (FinER & FormuLA datasets)

86.9%

Latency Reduction

Reduction in adaptation latency vs. prior prompt optimisation methods

** AppWorld is a benchmark that ranks AI agents based on how well they perform real-world, multi-step tasks across simulated applications such as email, calendars, and e-commerce systems. It evaluates not just whether a task is completed, but how accurately, efficiently, and safely the agent navigates complex workflows — measuring how effectively AI systems can operate as practical digital assistants.*

Case Study: ACE in Financial Analysis (XBRL)

The challenges ACE is designed to solve

Structured but Complex

Financial statements are broken into hundreds of tagged elements — many representing similar concepts across different reporting contexts. Variations across companies make consistent extraction difficult at scale.

e.g. A filing may include "Total Revenue," "Operating Revenue," and "Net Sales" — selecting the wrong tag distorts the analysis.

139 Fine-Grained Categories

Identifying and labeling financial entities across a large set of closely related categories. Mislabeling a single entity can propagate errors through all downstream analysis.

e.g. "Interest" could mean interest income, interest expense, or ownership interest — context determines everything.

Extract, Then Compute

Requires extracting specific values from structured filings and applying financial logic correctly. Errors in data selection or misunderstood variable relationships compound into wrong results.

e.g. Calculating profit fails if operating expenses are confused with total expenses.

Case Study: ACE in Financial Analysis (XBRL)

How ACE masters complex domain knowledge through division of labor

ACE addresses complex XBRL-based reasoning by dividing labor across three specialized roles. Researchers evaluated its performance on FiNER, which involves labeling 139 financial entity types, and FormuLA, which requires numerical computation from structured filings.

The Generator: Task Execution

Acts as the financial analyst. Attempts FiNER and FormuLA queries using the current playbook, producing a reasoning trace that exposes calculation methods and any misapplied XBRL rules.

The Reflector: Diagnosis & Insight Extraction

Reviews the execution trace to pinpoint exactly where the Generator went wrong. Distills errors into specific, actionable XBRL rules and financial concepts—not vague feedback.

The Curator: Knowledge Preservation

Files the extracted rules into permanent memory as incremental delta updates—never rewriting the full playbook. Builds a growing, precise XBRL rulebook over time.

8.6%

Avg. Performance Gain

Over strong baselines on FiNER and FormuLA benchmarks.

10.9%

Offline Adaptation Lead

Surpassed ICL, MIPROv2, and GEPA during offline adaptation.

6.2%

Online Adaptation Lead

Exceeded Dynamic Cheatsheet and other adaptive memory methods.

Case Study: AI-Powered Expense Management

Continuous improvement without costly redesign cycles

ACE refines expense categorization by learning from user corrections, such as reclassifying Uber from "Travel" to "Transport" without requiring manual system redesigns.

The Generator: Product Behavior

The AI categorizes transactions and generates spending insights using current rules, serving as the initial functional release.

The Reflector: Feedback & Diagnosis

Analyzes user corrections to identify patterns, such as ride-sharing merchants being misclassified 42% of the time.

The Curator: Improvement Engine

Merges insights into the playbook as incremental updates, evolving the system continuously without full rewrites.

Traditional Approach:

Collect feedback → redesign → release v2. Slow, lossy cycles.

ACE Approach:

Feedback is immediately structured and reused. Continuous improvement, no lost learning.

Criteria for Implementation

ACE is best piloted where the following conditions already exist: they define the scope where implementation is viable and measurable.

Condition	Why It Matters
Strong CI Coverage	Reliable, objective feedback signals for the Reflector to learn from
Repetitive Change Patterns	High-frequency tasks where accumulated playbook rules compound in value
Low Regulatory Sensitivity	Reduced compliance risk during the initial learning and trace-collection phase
Clear Rollback Mechanisms	Safe recovery path if an approved playbook rule produces an unexpected outcome
Willing Reviewers	Human oversight to validate lessons before they enter the active playbook

The initial objective is not autonomous deployment, it is proving that ACE reduces repeated mistakes and captures engineering knowledge in a reusable, auditable way.

Staged Rollout Model

Autonomy is earned incrementally: each level requires demonstrated reliability at the previous stage before progression is considered.

Level 0: Advisory Only: The agent proposes, and humans approve all actions. ACE collects traces only.

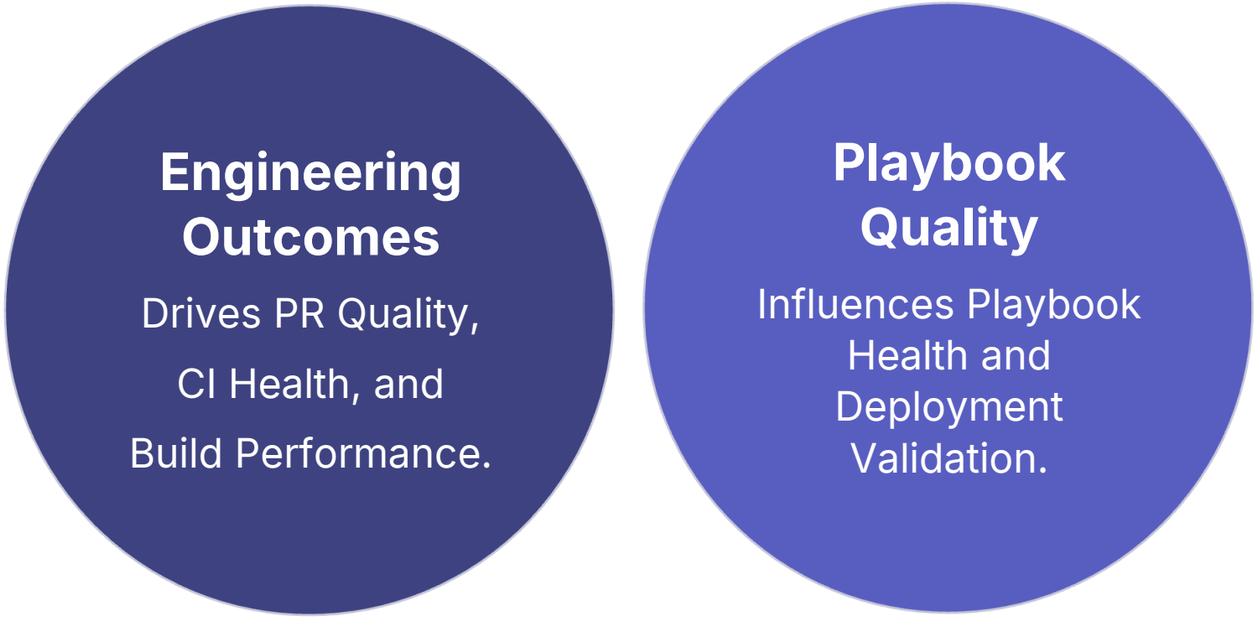
Level 1: Draft PR Mode: The agent opens PRs with risk notes but cannot merge.

Level 2: Scoped Auto-Merge: The agent merges only low-risk changes with green CI and no regulated modules.

Level 3: Near-Autonomous Rollout: The agent deploys within tightly bounded services using canary releases and automatic rollback.

Performance Metrics

Success of ACE framework is measured across two dimensions: the quality of engineering outcomes it produces, and the health of the playbook.



The ACE paper reports strong benchmark gains, but enterprise value is realized only when those gains translate into safer, cheaper, and faster engineering outcomes in production.

References

- Zhang, Q., et al. (2025). *Agentic Context Engineering: Evolving contexts for self-improving language models.* arXiv. ICLR 2026 submission. arXiv:2510.04618
- Shinn, N., et al. (2023). *Reflexion: Language agents with verbal reinforcement learning.* NeurIPS 36. arXiv:2303.11366
- Wang, X., et al. (2024). *Executable code actions elicit better LLM agents (CodeAct).* ICML. arXiv:2402.01030
- Opsahl-Ong, K., et al. (2024). *Optimizing instructions and demonstrations for multi-stage language model programs (MIPRO).* EMNLP 2024. ACL Anthology
- Suzgun, M., et al. (2025). *Dynamic Cheatsheet: Test-time learning with adaptive memory.* arXiv. arXiv:2504.07952
- Jiang, P., et al. (2025). *Adaptation of agentic AI.* arXiv. arXiv:2512.16301